be set—for example, because user_id is faster to access than user—this may cause problems in your tests. (I suppose you can have an ID as one of the key/value pairs when creating the subordinate object. I've never tried that myself, but it could work.)

My preferred strategy is to not specify attributes in factories at all. If I need associated objects in a specific test, I explicitly add them to the test at the point they're needed.

Why?

- The surest way to keep factory_bot from creating large object trees is to not define large object trees.
- Tests that require multiple degrees of associated objects often indicate improperly factored code. Making it harder to write associations in tests nudges me in the direction of code that can be tested without associations.

Prescription 18

Avoid defining associations automatically in factory_bot definitions. Set them test by test, as needed. You'll wind up with more manageable test data.

The only downside is that there's a little more typing involved in some tests. Especially in Rails 5, where by default belongs_to associations are required, when saving data you may need to create objects just to make ActiveRecord happy. That's another argument against saving data unless you need to.

Managing Duplication in Factories

Once you have more than a couple of factories in your application, you want to make sure you can manage complexity and duplication. Factory_bot has a number of features to allow you to do just that.

Sequences

A common problem is the creation of multiple objects that require unique values. This most often happens with unique user attributes such as a login or email address. To allow the easy creation of unique attributes, factory_bot allows you to define an attribute as part of a sequence of values. The short version of the syntax looks like this:

```
FactoryBot.define do
  factory :task do
    sequence(:title) { |n| "Task #{n}" }
  end
end
```